

Como construir apps com IA: a lógica por trás do que fizemos e o que aprender para os próximos

Explicação do Medde por dentro, erros que custaram retrabalho, como pedir melhor, trilha de estudo, fontes e livros.

Versão de 08/10/2026.

Para quem é

Para você, que construiu um app de verdade (com login, banco de dados, voz por IA, notificações, painel, loja e publicação) sem ter começado como programador. Você não precisa virar programador. Precisa aprender a **pensar como quem dirige a construção**: decidir o que vale a pena, explicar com clareza, conferir o resultado e saber o que perguntar.

A ideia central deste material

Quem usa IA para construir é o **dono do produto** e o **fiscal da obra**. A IA é o time de construção muito rápido, mas que só faz bem o que foi bem descrito e que não vê o seu celular, os seus clientes nem o seu dinheiro. Quanto melhor você decide e descreve, menos retrabalho.

Nota de honestidade: as fontes e livros citados são conhecidos e confiáveis, mas eu não consigo conferir daqui se cada link, edição em português ou preço ainda existe. Confira antes de comprar.

Como usar este material

Parte	O que é	Quando ler
A	O mapa do Medde: como o app funciona por dentro, em linguagem simples	Primeiro
B	Os conceitos, um por um: o que é, onde aparece no Medde, o que pesquisar	Aos poucos, como consulta
C	O que fizemos e onde houve retrabalho (com a causa real e a lição)	Depois da A
D	Como pedir melhor: fórmula, modelos prontos e erros comuns	Antes do próximo projeto
E	Processo para não refazer: do problema ao app publicado	Antes do próximo projeto
F	Plano de estudo em 90 dias, com horas por semana	Para organizar o aprendizado
G	Exercícios usando o próprio Medde	Para praticar
H	Fontes confiáveis, livros e glossário	Consulta

Como aprender de verdade

1) Leia um tema. 2) Procure o mesmo tema no Medde (eu digo onde). 3) Peça à IA: *"explique de novo como se eu tivesse 12 anos e depois me faça 5 perguntas para ver se entendi"*. 4) Escreva com as suas palavras. Quem consegue explicar o conceito para outra pessoa aprendeu.

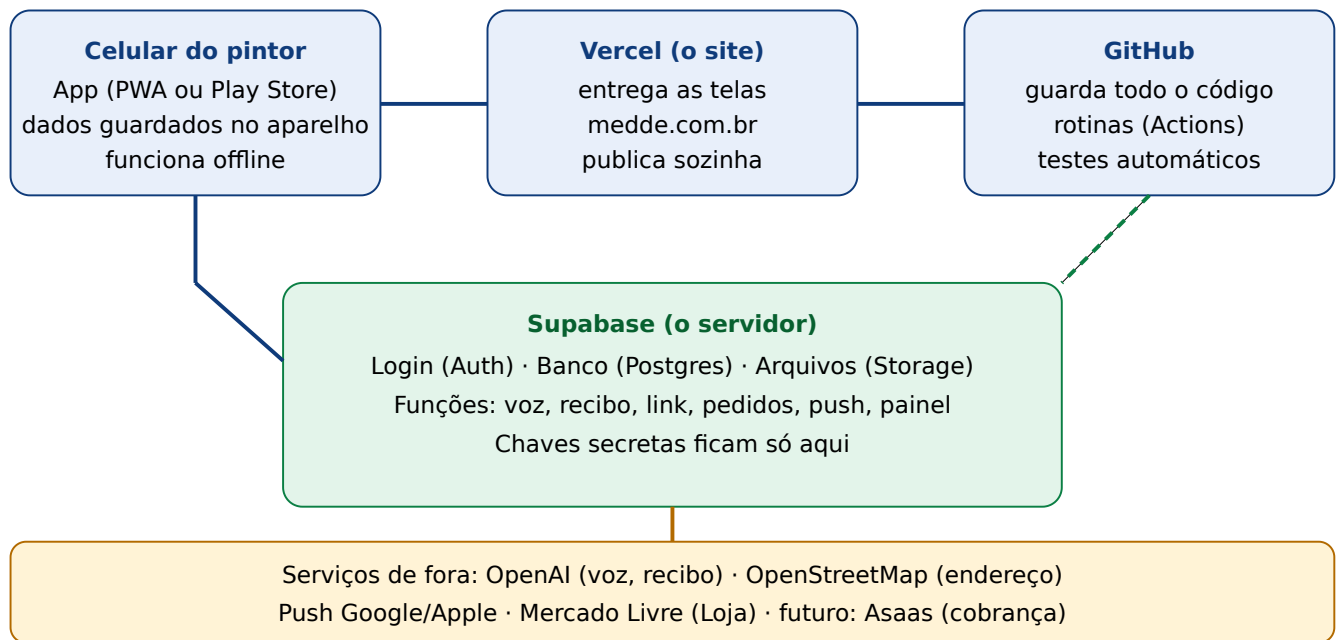
Parte A. O mapa do Medde: como ele funciona por dentro

A.1 A imagem geral

Pense no Medde como um **restaurante**. O cliente (o pintor) senta numa mesa e faz o pedido. Há um salão, uma cozinha, uma despensa e fornecedores.

Restaurante	Medde	O que é na prática
Salão e cardápio	Interface (as telas)	O que o pintor vê e toca no celular. Feito com Next.js e React, hospedado na Vercel em <code>medde.com.br</code> . Pastas <code>src/app</code> e <code>src/components</code> .
Anotações do garçom	Dados no aparelho	O app anota tudo primeiro no próprio celular (<code>localStorage</code>), por isso funciona com internet fraca. Pasta <code>src/repositories</code> (<code>localStorage.ts</code>).
Cozinha (receitas)	Regras de negócio	As contas e regras: preço por m ² , status automático do orçamento, plano de pagamento. Pastas <code>src/engine</code> (cálculo puro) e <code>src/modules</code> (casos de uso).
Despensa central	Banco de dados (Supabase)	Onde a conta de cada pintor fica guardada de forma segura e acessível de qualquer aparelho. Tabelas como <code>user_data</code> e <code>subscriptions</code> .
Segurança na porta	Login e permissões	Só o dono vê os próprios dados (regra chamada RLS) e o painel só abre para o administrador.
Cozinheiros especiais	Funções do servidor (Edge Functions)	Pequenos programas que rodam no servidor, onde as chaves secretas ficam seguras: voz, recibo, link do orçamento, notificações, painel.
Fornecedores	Serviços externos	OpenAI (voz e recibo), OpenStreetMap (endereço), serviço de push do Google/Apple (notificações), Mercado Livre (Loja).
Mestre de obras	Publicação automática	GitHub guarda o código; a cada mudança aprovada, a Vercel e a rotina "Supabase deploy" colocam no ar sozinhas.

A.2 Diagrama



Setas cheias: o app conversa com o site (para carregar) e com o Supabase (para guardar dados e usar funções). A linha tracejada: o GitHub publica as funções e roda o banco no Supabase por rotinas.

A.3 O caminho de um orçamento (do toque ao WhatsApp do cliente)

1. O pintor faz a visita: o app guarda tudo no **celular** (rápido, offline). Fotos e áudios ficam em arquivos no aparelho.
2. Quando há internet, o app envia uma cópia para o **Supabase** (tabela `user_data`, um documento por conta). Isso se chama **sincronização** (`src/repositories/sync.ts` e `cloudStore.ts`).
3. O pintor monta o orçamento. O **motor de cálculo** (`src/engine`) soma áreas, serviços, materiais, margem. Ele não conhece telas nem banco: só recebe números e devolve números. Por isso é fácil de testar e confiar.
4. "Enviar pelo WhatsApp": o app pede à função `quote-link` para criar um **link público** com uma cópia limpa do orçamento (sem custos nem lucro).
5. O cliente abre o link. A função registra a visualização e chama a função `push`, que manda a **notificação** "o cliente abriu" para o celular do pintor.
6. O pintor marca "Fechou": o app cria a **obra** e passa a acompanhar pagamentos e gastos.
7. Você, no painel, vê os totais de todas as contas pela função `admin-stats`.

A.4 Por que decidimos assim (as grandes escolhas)

1. "Dados primeiro no aparelho" (local-first)

O pintor está na obra, com sinal ruim. Se o app dependesse da internet para cada toque, travaria. Guardar primeiro no celular e sincronizar depois resolve. *Custo: é preciso cuidar de conflitos e do que ainda não foi enviado (por isso o painel pode contar a menos).* **Pesquisar:** "local-first software", "offline first PWA".

2. Um documento por conta (JSON) em vez de muitas tabelas

No começo as regras mudavam toda semana. Um documento flexível evita mexer no banco a cada ideia. *Custo: é difícil fazer relatórios entre contas (o painel precisa ler documentos grandes). Quando o produto estabilizar, pode ser melhor ter tabelas separadas.* **Pesquisar:** "JSONB PostgreSQL", "normalização de banco de dados".

3. Motor de cálculo puro, separado das telas

Preço errado destrói a confiança. Um motor sem tela nem banco tem testes que garantem a conta. A regra do repositório impede que as telas "pulem" para o banco ou o cálculo (arquivo `eslint.config.mjs`). **Pesquisar:** "arquitetura em camadas", "separação de responsabilidades", "funções puras".

4. Chaves secretas só em funções do servidor

Tudo que está no app pode ser lido por quem abrir o navegador. Por isso a chave da OpenAI e a chave "administrador" do Supabase ficam só nas Edge Functions e nos Secrets do GitHub. **Pesquisar:** "secrets management", "OWASP Top 10", "service role key Supabase".

5. Publicação automática e testes

Cada mudança passa por testes antes de ir ao ar (`ci.yml`). Se algo quebrar, você sabe na hora. *Lição: durante dias o CI ficou vermelho por um aviso de lint e ninguém olhou. Regra nova: nunca ignore o vermelho.* **Pesquisar:** "CI/CD", "GitHub Actions".

6. PWA em vez de app nativo primeiro

Um só código funciona em Android, iPhone e computador, e atualiza sem loja. O app da Play Store é uma "casca" (TWA) que abre o site. *Custo: acesso limitado a alguns recursos do celular e depende do navegador.* **Pesquisar:** "PWA", "Trusted Web Activity", "Digital Asset Links".

7. Painel via função de servidor, não direto do app

O painel precisa ler dados de todas as contas. Isso exige uma chave poderosa, que nunca pode ir ao navegador. Então o painel pede à função `admin-stats`, que confere se é você e devolve só totais. **Pesquisar:** "autorização vs autenticação", "princípio do menor privilégio".

A.5 Mapa das pastas do projeto

Pasta	O que tem
<code>src/app</code>	As telas (uma pasta por rota: visitas, orcamentos, obras, loja, painel, privacidade...)

<code>src/components</code>	Peças reutilizáveis: botão, cartão, blocos recolhíveis, gravação de áudio, fotos. O arquivo <code>ui.tsx</code> é o "kit de peças" visual.
<code>src/modules</code>	Regras do app: visitas, orçamentos, obras, status automático, assinatura, push, painel.
<code>src/engine</code>	Cálculo do orçamento (puro, com testes).
<code>src/repositories</code>	Acesso a dados: aparelho, nuvem, arquivos.
<code>src/shared</code>	Coisas comuns: dinheiro, formatação, marca.
<code>supabase/functions</code>	Funções do servidor (uma pasta por função, com <code>index.ts</code> e <code>logic.ts</code> testável).
<code>supabase/migrations</code>	Arquivos SQL numerados que criam e mudam tabelas (0001, 0002...). Rodam na ordem e uma vez só.
<code>tests</code>	Testes de tela com navegador automático (Playwright), que usam o app como uma pessoa usaria.
<code>docs</code>	A "memória" do projeto: decisões, guias, textos jurídicos. Mantenha atualizada: é o que permite outro assistente continuar.
<code>.github/workflows</code>	As rotinas automáticas (testes, deploy, backup, anti-pausa).
<code>public</code>	Arquivos servidos como estão: ícones, <code>sw.js</code> (service worker), <code>.well-known/assetlinks.json</code> .

Parte B. Os conceitos, um por um

Cada conceito tem: **O que é** **No Medde** **Pesquisar** . Comece pelos marcados com ★ (os que mais evitam retrabalho).

B.1 Fundamentos da web

★ Cliente, servidor e internet

O que é O celular (cliente) pede coisas; o servidor responde. Tudo é troca de pedidos e respostas por endereços (URLs). **No Medde** O app (cliente) pede dados ao Supabase (servidor). **Pesquisar** "o que acontece quando você digita um endereço no navegador", "HTTP request response".

HTML, CSS e JavaScript

O que é HTML é a estrutura, CSS é a aparência, JavaScript é o comportamento. **No Medde** Tudo é escrito em TypeScript (JavaScript com checagem de erros) e Tailwind (CSS por classes). **Pesquisar** "MDN HTML básico", "flexbox", "grid", "TypeScript para iniciantes".

★ API e JSON

O que é API é o "balcão" onde um programa pede serviço a outro. JSON é o formato do papel que passa pelo balcão (`{"nome":"Ana"}`). **No Medde** O app chama as funções do Supabase enviando e recebendo JSON. **Pesquisar** "REST API explicada", "JSON".

Componentes e estado (React)

O que é A tela é montada com peças (componentes). "Estado" é a memória da tela (aba aberta, texto digitado). **No Medde** O `BlockRecolhive1` guarda só em memória se está aberto. Fechar e abrir a tela volta ao padrão. **Pesquisar** "React estado e props", "react.dev learn".

B.2 Dados e segurança

★ Banco de dados, tabelas e SQL

O que é Um banco é um conjunto de planilhas (tabelas) com regras. SQL é a linguagem para perguntar e alterar. **No Medde** `subscriptions` guarda a assinatura de cada conta. As migrações `0003` , `0010` criaram tabelas. **Pesquisar** "SQL básico select insert update", "SQLBolt", "PostgreSQL tutorial".

★ Migrações

O que é Arquivos SQL numerados que levam o banco do estado A ao B de forma repetível. **No Medde** Esquecemos de rodar a `0003` : o painel pediu uma tabela que não existia. **Pesquisar** "database migrations", "schema versioning". **Lição:** toda mudança de banco precisa estar numa lista de "já rodado ou não".

★ Autenticação x autorização

O que é Autenticação = provar quem você é (login). Autorização = o que você pode fazer. **No Medde** Login por e-mail e senha (Supabase Auth); o painel só abre para o e-mail do administrador. **Pesquisar** "authentication vs authorization", "JWT explicado".

★ RLS (segurança por linha)

O que é Regra no banco: "este usuário só enxerga as linhas dele". **No Medde** Cada tabela tem RLS ligado. Em `user_data`, cada conta só lê a própria linha. Tabelas sem política (como `admin_settings`) só as funções do servidor acessam. **Pesquisar** "Supabase Row Level Security".

★ Chaves, tokens e segredos

O que é Senhas que programas usam entre si. Há as **públicas** (podem ir no app: publishable key) e as **secretas** (só no servidor: service role, OpenAI, token do Supabase). **No Medde** Ficam em **Secrets** do GitHub e do Supabase. Nunca no chat, código ou print. **Pesquisar** "OWASP Top 10", "secrets management", "o que fazer se vazou uma chave". **Regra:** vazou, revogue e crie outra.

CORS

O que é Regra do navegador: um site só chama servidores que autorizaram esse site. **No Medde** Cada função lista os endereços permitidos (`ALLOWED`). Se adicionar o `admin.medde.com.br`, ele precisa estar na lista. **Pesquisar** "CORS explicado".

Webhook

O que é O serviço de pagamento avisa o seu servidor ("pagou!") chamando um endereço seu. **No Medde** Será a função que atualiza a assinatura. **Pesquisar** "webhook explicado", "assinatura de webhook".

LGPD

O que é A lei brasileira de proteção de dados. Exige base legal, transparência (política de privacidade), segurança e direitos do titular (como excluir). **No Medde** Exclusão de conta, política, texto para o advogado, cuidado com dados dos clientes dos pintores. **Pesquisar** "LGPD para startups", "ANPD guia", "controlador e operador".

B.3 Construção e publicação

★ Git e GitHub

O que é Git guarda o histórico de todas as versões do código. GitHub é o site onde fica guardado e onde as rotinas rodam. **Branch** é uma cópia de trabalho; **commit** é um ponto salvo com mensagem; **main** é a versão oficial. **No Medde** Trabalhamos num branch e só vai para a `main` quando você diz "pode publicar". **Pesquisar** "Pro Git livro", "git branch commit merge para iniciantes".

★ CI/CD, testes e lint

O que é CI = a cada mudança, rodar testes automáticos. CD = publicar automaticamente. Lint = corretor ortográfico do código. **No Medde** 217 testes de lógica (Vitest) + testes de tela (Playwright) + lint. Verde = pode confiar mais. **Pesquisar** "testes unitários vs testes de ponta a ponta", "GitHub Actions".

Ambientes e variáveis de ambiente

O que é O mesmo app roda em lugares diferentes (seu computador, produção) com configurações diferentes, guardadas em variáveis. **No Medde** `NEXT_PUBLIC_SUPABASE_URL` liga a nuvem; sem ela o app roda só local. É por isso que há "build local" e "build de nuvem" nos testes. **Pesquisar** "environment variables", "12-factor app".

Deploy, rollback e versões

O que é Deploy é colocar no ar. Rollback é voltar para a versão anterior. **No Medde** Vercel guarda cada versão e permite "promover" uma antiga. **Pesquisar** "deploy rollback Vercel".

PWA, service worker, TWA

O que é PWA é um site que se comporta como app (instalável, notificações). Service worker é o ajudante nos bastidores. TWA é o jeito de colocar o PWA na Play Store. **No Medde** `public/sw.js`, `manifest.ts`, PWABuilder, `assetlinks.json`. **Pesquisar** "web.dev learn PWA", "Digital Asset Links".

Notificações push

O que é O servidor pede ao Google/Apple para acordar o aparelho com uma mensagem. Usa um par de chaves (VAPID). **No Medde** Função `push`, tabelas `push_keys` e `push_subscriptions`. **Pesquisar** "Web Push VAPID".

B.4 Produto e experiência

★ Fluxo do usuário e telas (wireframe)

O que é Um desenho simples (no papel) de cada tela e do caminho entre elas, **antes** de construir. **No Medde** Refizemos telas várias vezes porque o desenho veio depois (Parte C). **Pesquisar** "wireframe", "user flow", "Figma para iniciantes", "jornada do usuário".

★ Divulgação progressiva e simplicidade

O que é Mostrar só o necessário e deixar o resto um toque adiante. **No Medde** Blocos recolhíveis, abas, deslizar, botão principal único por tela. **Pesquisar** "progressive disclosure", "Lei de Hick", "heurísticas de Nielsen".

Sistema de design (tokens, componentes)

O que é Regras visuais reutilizáveis (cores, fontes, espaços) e peças prontas. **No Medde** Cores e fontes em `globals.css`, peças em `ui.tsx`. Quando mudamos o visual leve, mudamos nas peças e as telas acompanharam. **Pesquisar** "design tokens", "atomic design", "Refactoring UI".

Acessibilidade

O que é Fazer o app usável por todos: contraste, tamanho de toque (48 px), texto mínimo, leitores de tela. **No Medde** Auditoria automática de toque e texto; `aria-label` nos botões só com ícone. **Pesquisar** "WCAG 2.2", "acessibilidade mobile".

★ Métricas de produto

O que é Números que dizem se o produto funciona: ativação, retenção, conversão, cancelamento. **No Medde** O painel: funil, contas ativas e sumidas, assinantes, custo de IA. **Pesquisar** "métricas AARRR pirate metrics", "retenção", "churn", "MRR", "CAC e LTV".

Parte C. O que fizemos e onde houve retrabalho

Retrabalho não é fracasso: é informação. Cada caso abaixo aconteceu de verdade neste projeto. A coluna "causa" é o que mais importa: quase sempre é **uma decisão que faltou antes de construir**.

O que aconteceu	Causa real	Lição e como evitar
Criamos guias de uso ("tours") e depois removemos todos	Construímos antes de testar se o pintor precisava deles.	Teste a ideia com 3 pintores usando um protótipo (ou só o desenho) antes de programar. Pesquisar: "protótipo de baixa fidelidade", "teste de usabilidade com 5 usuários".
Tela de orçamento simplificada em várias rodadas (sem "Ver PDF", botão único, ícones)	Cada ajuste veio depois de olhar a tela pronta no celular. Faltou um desenho com a decisão "uma ação principal por tela".	Defina por tela: qual é a ação principal, o que é secundário, o que pode sumir . Escreva isso antes.
Visitas: 3 abas viraram 2 abas mais um botão	Não tínhamos definido o que o pintor faz na primeira abertura do dia.	Escreva o "primeiro minuto" de cada perfil: o que ele quer ver ao abrir o app? Pesquisar: "jobs to be done".
Redesenho "leve" de todas as telas, depois de tudo pronto (50 telas, centenas de testes ajustados)	O visual foi decidido por último. Mudar o visual mexeu em quase tudo.	Decida o sistema visual (cores, fontes, tamanhos, peças) primeiro . Depois as telas só o usam. Foi o que permitiu mudar rápido: o <code>ui.tsx</code> concentrou as peças. Pesquisar: "design system", "design tokens".
Painel mostrou erro "tabela subscriptions não encontrada"	A migração <code>0003</code> estava no repositório mas nunca foi rodada no banco. O documento dizia "falta confirmar" e ninguém conferiu.	Mantenha uma lista de estado do ambiente : "migração X rodada em tal dia". Antes de usar uma tabela nova, confira se existe. Pesquisar: "database migrations".
Token do Supabase e senha do banco apareceram no chat e em prints	Pedimos o valor onde a ação exigia "colar", sem um passo seguro desenhado desde o começo.	Desenhe o caminho das chaves antes: elas vão direto para o campo de Secrets , nunca por mensagem. Se vazou: revogue e crie outra no mesmo dia.
A verificação automática (CI) ficou vermelha por vários dias	Um aviso de lint antigo, sem impacto no app, mas ninguém olhou o vermelho.	Vermelho nunca é "normal". Combine: não publicar com CI vermelho . Peça o status do CI em toda entrega.

Arquivo de teste temporário foi parar no <code>main</code>	Um comando travou antes de apagar o arquivo, e o commit pegou tudo.	Peça sempre: <i>"mostre a lista de arquivos do commit antes de publicar"</i> . Revisar o que vai junto é barato.
Valores em reais cortados no painel (só apareceu no seu celular)	O teste automático usava dados curtos; os seus eram maiores.	Teste com dados reais e extremos (nomes e números longos), e peça um print no celular depois de cada publicação visual.
Testes de tela quebraram quando as informações passaram a ficar dentro de blocos fechados	Os testes procuravam coisas que agora ficam escondidas até tocar.	É normal: mudou o comportamento, mudam os testes. O ponto é perguntar sempre " o que isto quebra? " antes de aprovar uma mudança grande.
Build "de nuvem" e "local" diferentes	O app muda de comportamento conforme variáveis de ambiente.	Aprenda a diferença entre os ambientes e peça que os testes digam qual build usam .

O que deu muito certo (repita)

1) O pedido do designer: tinha **regra de ouro** (nada funcional muda), **lista fechada** do que podia mudar e **prova** (comparar antes e depois). 2) Publicar só depois do seu "pode mover/publicar". 3) Guardar decisões em `docs/`. 4) Testes automáticos que pegaram erros antes de você. 5) Pedir o passo a passo em imagens quando você estava no celular.

Parte D. Como pedir melhor à IA (prompts)

D.1 A fórmula: OBJETIVO, CONTEXTO, REGRAS, PRONTO QUANDO

Parte	Pergunta que responde	Exemplo do Medde
1. Objetivo	O que você quer que exista no fim, em uma frase, do ponto de vista de quem usa?	"O pintor vê, num lugar só, quais clientes pararam de usar o app, com telefone para chamar."
2. Contexto	Quem usa, onde, com que aparelho? O que já existe? O que já tentamos?	"Só eu uso, no celular. Já existe a função admin-stats e a tela /painel."
3. Regras e limites	O que NÃO pode mudar? O que é obrigatório? Prazo? Custo?	"Não mostrar dados dos clientes dos pintores. Sem serviços pagos novos."
4. Pronto quando	Como saberemos que deu certo? O que você quer ver para acreditar?	"Testes passando, um print no meu celular, e me avise antes de publicar."

Por que isso evita retrabalho

A IA preenche lacunas com suposições. Cada parte em branco vira uma suposição. Se a suposição estiver errada, você só descobre depois de pronto. A fórmula força você a decidir **antes**.

D.2 Modelos prontos (copie e adapte)

Nova funcionalidade

```
OBJETIVO: [o que o usuário consegue fazer ao final, em uma frase]
QUEM USA E ONDE: [perfil, aparelho, situação (ex.: na obra, com uma mão)]
HOJE: [o que existe e o que incomoda]
REGRAS: [o que não pode mudar; dados que não podem aparecer; custo máximo]
DECISÕES JÁ TOMADAS: [liste]
DÚVIDAS QUE VOCÊ (IA) DEVE ME FAZER ANTES DE COMEÇAR: [ou "faça no máximo 5 perguntas"]
PRONTO QUANDO: [critérios que eu consigo conferir: tela, print, teste, número]
ENTREGA: primeiro um plano curto para eu aprovar; só depois construa;
no fim me diga o que mudou, o que pode quebrar, e como desfazer.
```

Corrigir um erro

```
O QUE EU FIZ: [passos exatos, ex.: "abri medde.com.br/painel logado"]
O QUE ESPERAVA: [...]
O QUE ACONTECEU: [texto do erro e print]
ONDE: [aparelho, navegador, conta, horário]
JÁ TENTEI: [...]
Antes de mudar código: explique a causa provável e como você vai confirmar.
```

Mudança visual

TELA: [nome]
HOJE: [print] DEPOIS (referência): [print ou desenho]
NÃO MUDAR: funções, textos, rotas e dados.
PODE MUDAR: [lista fechada de textos, cores, tamanhos]
PROVA: capture antes e depois com todos os blocos abertos e compare;
liste qualquer diferença que não esteja na lista permitida.

Decidir entre opções

Preciso decidir [X]. Liste 2 ou 3 opções com: vantagem, desvantagem, custo, risco, quanto trabalho dá e o que fica difícil de mudar depois.
Dê uma recomendação para o meu caso (leigo, sozinho, poucos usuários) e diga o que faria você mudar de ideia.

Revisão antes de publicar

Antes de eu autorizar, me mostre: (1) lista de arquivos alterados, (2) o que muda para o usuário, (3) o que pode quebrar e como saber, (4) resultado de testes e do CI, (5) como voltar atrás. Só publique quando eu escrever "pode publicar".

Aprender com a IA

Explique [conceito] como se eu fosse leigo, usando um exemplo do app Medde.
Depois me faça 5 perguntas (da mais fácil à mais difícil) e corrija minhas respostas.
Por fim, diga o que devo pesquisar em fontes confiáveis para me aprofundar.

D.3 Erros comuns de pedido (e o conserto)

Pedido fraco	Por que dá retrabalho	Melhor
"Deixa mais bonito"	Bonito para quem? Em que tela? Sem critério, a IA chuta.	"Reduza o peso visual da tela X: títulos 24 px, botões secundários sem borda; não mude textos."
"Faz um painel"	Quais números? Para quem? Com que dados?	Primeiro liste as decisões que o painel deve ajudar a tomar (foi o que fizemos).
"Está errado" (sem print)	A IA não vê o seu celular.	Print, passos, o esperado e o obtido.
Pedir cinco coisas numa mensagem	Mistura prioridades e esconde conflitos.	Uma entrega por vez, com critério de pronto, ou numerar e priorizar.
Mudar de ideia no meio sem avisar	O que foi feito vira lixo.	"Mudança de decisão: antes X, agora Y. Revise o que já foi feito."

Aceitar "pronto" sem prova	A IA pode errar com confiança.	Peça evidência : testes verdes, print, comparação antes/depois, lista de arquivos.
Colar senhas ou chaves no chat	Risco de segurança real.	Peça "me diga onde guardar a chave com segurança".

D.4 Como conferir o trabalho da IA sem saber programar

- **Peça o resumo em português** do que mudou e o impacto para o usuário.
- **Verifique no aparelho real**, não só nos testes. Teste com dados longos e com internet ruim.
- **Olhe os sinais**: CI verde, rotinas verdes, Vercel "Ready", painel mostrando números coerentes.
- **Pergunte "o que mais isto afeta?" e "como eu desfaço?"**.
- **Desconfie de certezas sem prova**. Frase útil: *"Como você sabe? Mostre."*
- **Mudanças pequenas e frequentes** são mais fáceis de conferir do que uma grande.
- Se algo importar muito (dinheiro, segurança, lei), peça **segunda opinião** (outro assistente ou um profissional) antes de confiar.

D.5 Memória do projeto: faça a IA lembrar por você

Assistentes esquecem entre conversas. O que não esquece é o repositório. Crie e mantenha:

- `docs/DECIS0ES.md` : lista de decisões com data e motivo ("Imposto 6% até o contador confirmar").
- `docs/ESTADO.md` : o que está ligado e rodado (migrações, secrets, domínios, versões na loja).
- `docs/FILA_DE_TRABALHO.md` : o que falta, em ordem (já existe).
- Um arquivo `CLAUDE.md` (ou equivalente) na raiz com: o que é o projeto, como rodar, regras do dono ("publicar só com 'pode publicar'", "nunca pedir chaves no chat"). Os assistentes de programação leem esse arquivo sozinhos. Veja a documentação do Claude Code.

Parte E. O processo para não refazer

E.1 O ciclo (da ideia ao app publicado)

#	Etapa	O que fazer	Saída
1	Problema	Converse com 5 a 10 pessoas do público. Pergunte sobre o passado ("como foi o último orçamento que você fez?"), não sobre o futuro ("você usaria?").	Frase do problema + dores reais
2	Quem e quando	Descreva o usuário principal e o momento de uso (na obra, no celular, com sinal fraco).	Perfil de 5 linhas
3	Fatia mínima	Qual a menor versão que já resolve a dor de uma pessoa? Corte o resto para "depois".	Lista "agora / depois / nunca"
4	Desenho	Desenhe as telas e o caminho (papel, Figma ou peça à IA um protótipo). Valide com 3 pessoas.	Wireframes aprovados
5	Regras escritas	Cálculos, estados, exceções. ("Orçamento aberto vira perdido 14 dias depois da validade.")	<code>docs/DECIS0ES.md</code>
6	Construir pequeno	Uma fatia por vez, com critério de pronto e prova. Publicar só com "pode publicar".	Entregas pequenas
7	Testar com gente real	Roteiro de teste, grupo de WhatsApp, observe sem ajudar.	Lista de problemas
8	Medir	Painel: quem ativou, voltou, pagou, sumiu.	Decisões baseadas em números
9	Repetir	Volte ao passo 3 com o que aprendeu.	Próxima fatia

E.2 Decisões que só você pode tomar (a IA não deve decidir sozinha)

- Quem é o usuário e qual dor resolvemos primeiro.
- Quanto cobrar, para quem e como.
- O que o app **não** vai fazer.
- Que dados coletar e o que prometer sobre eles (privacidade).
- Quando publicar e para quem.
- Quanto gastar por mês.

E.3 Checklists curtos

Antes de pedir uma funcionalidade

- ☐ Escrevi o objetivo em uma frase
- ☐ Sei quem usa e onde
- ☐ Listei o que não pode mudar
- ☐ Defini como vou saber que deu certo
- ☐ Cortei o que pode esperar
- ☐ Tenho print ou desenho de referência

Antes de autorizar a publicação

- ☐ CI verde
- ☐ Vi no celular real
- ☐ Li a lista de arquivos alterados
- ☐ Sei como voltar atrás
- ☐ Nenhuma chave ou dado pessoal no código
- ☐ A documentação foi atualizada

Definição de "pronto"

Funciona no celular real · testes passando · casos de erro tratados (sem internet, campo vazio, texto muito longo) · acessível (toque 48 px, texto 16 px) · documentado · você conseguiu explicar para alguém.

Quando algo quebrar

1) Respire. 2) Anote o que mudou por último. 3) Vercel: promover a versão anterior. 4) Peça diagnóstico antes de correção. 5) Depois, escreva a lição em `docs/DECIS0ES.md`.

Parte F. Plano de estudo em 90 dias

Ritmo sugerido

5 a 6 horas por semana (3 sessões de 1h30 a 2h). Meta: **entender o suficiente para dirigir a IA, revisar o trabalho dela e não ser surpreendido**. Não precisa decorar nada: precisa saber o que existe, o que perguntar e onde procurar.

Período	Foco	Tópicos para pesquisar (nessa ordem)	Resultado
Semanas 1 a 3	Como a web funciona	cliente e servidor · HTTP · URL e domínio · DNS · HTML, CSS e JavaScript (o que cada um faz) · JSON · API · navegador (DevTools: aba Rede e Console)	Você explica o que acontece quando abre medde.com.br
Semanas 4 a 6	Git, GitHub e publicação	repositório · commit · branch · merge · pull request · main · GitHub Actions (ler uma rotina) · Vercel (deploys, ambientes) · variáveis de ambiente	Você lê o histórico de commits e entende o que mudou
Semanas 7 a 9	Dados e segurança	SQL básico (select, insert, update, where, join) · Postgres · tabelas e chaves · migrações · Supabase Auth · JWT · RLS · secrets · CORS · OWASP Top 10 (visão geral)	Você lê uma migração (ex.: 0010) e explica cada linha
Semanas 10 a 11	Front-end moderno	TypeScript (tipos) · React (componentes, estado, props) · Next.js (rotas, App Router) · Tailwind · PWA (manifest, service worker)	Você identifica onde fica cada tela e muda um texto com segurança
Semana 12	Qualidade	testes unitários e de ponta a ponta · lint · CI · acessibilidade (WCAG, contraste, toque) · desempenho no celular	Você lê o resultado do CI e sabe o que cada parte checa
Paralelo (todas as semanas)	Produto e negócio	entrevistas com usuários · jobs to be done · funil e retenção · precificação de SaaS · LGPD · regras da Google Play · atendimento e suporte	Um caderno de aprendizados dos testes com pintores

Trilha mínima se você tiver pouco tempo (20 horas no total)

1. Como a web funciona (3h).
2. Git e GitHub: o básico (2h).
3. SQL e RLS (5h).
4. Segurança de chaves e OWASP resumido (2h).
5. Noção de React e componentes (3h).
6. Testes e CI (2h).
7. Métricas de produto (3h).

Como aprender mais rápido usando a IA como professora

- Peça para ela **ler um arquivo do Medde e explicar linha por linha** em linguagem simples.
- Peça **exercícios** baseados no seu próprio projeto.
- Peça "**o que posso estar entendendo errado?**" depois de explicar com as suas palavras.
- Peça **fontes oficiais** e confira se existem. Se não achar o link, desconfie.

Parte G. Exercícios práticos usando o Medde

Aprender fazendo no seu próprio projeto fixa mais do que qualquer curso. Todos estes são pequenos e seguros (use um branch ou peça que a IA faça com você olhando).

#	Exercício	O que você aprende
1	Abra <code>src/modules/store.ts</code> e adicione um item novo na Loja ("Rolo de espuma").	Estrutura de dados, editar pelo GitHub, deploy automático.
2	Mude a frase de apoio da tela Loja e veja no ar em 5 minutos.	Ciclo editar → CI → Vercel.
3	Abra <code>supabase/migrations/0010_admin_settings.sql</code> e explique cada linha com as suas palavras.	Tabelas, tipos, RLS, valores padrão.
4	No Supabase, abra o Table Editor e veja a tabela <code>subscriptions</code> . Ache a sua conta.	Tabelas reais e a relação com usuários.
5	Leia <code>src/modules/autoStatus.ts</code> e descreva em 5 linhas quando um orçamento vira "perdido sozinho".	Regras de negócio escritas em código.
6	Rode a rotina "Supabase keepalive" e leia o log dela.	Ler o log de uma rotina.
7	Provoque um erro de propósito (peça à IA um teste que falhe) e veja o CI ficar vermelho; depois conserte.	Como o CI protege você.
8	Desenhe no papel uma tela nova (ex.: "Indique um pintor") e escreva as regras antes de pedir.	Wireframe e critério de pronto.
9	Escreva um "pedido" usando o modelo D.2 para uma melhoria pequena e peça à IA para criticar o seu pedido antes de executar.	Boa descrição de tarefa.
10	Pesquise o que é Row Level Security e explique por que um pintor não vê os dados de outro.	Autorização na prática.

-
- | | | |
|----|--|-------------------------|
| 11 | Entreviste 3 pintores com perguntas sobre o passado e anote 5 dores novas. | Descoberta de problema. |
|----|--|-------------------------|
-
- | | | |
|----|---|----------------------|
| 12 | Olhe o painel por 7 dias e escreva 3 decisões que ele sugere. | Decidir com números. |
|----|---|----------------------|
-

Parte H. Fontes confiáveis, livros e glossário

Antes de usar

Links e edições mudam. Todos abaixo são fontes oficiais ou muito conhecidas, mas eu não consigo abrir cada uma daqui. Se um link não abrir, pesquise o nome entre aspas. Prefira sempre a **documentação oficial** e conteúdo recente (a tecnologia muda rápido).

H.1 Documentação oficial (grátis)

Tema	Fonte
Web (HTML, CSS, JS)	MDN Web Docs (em português): developer.mozilla.org/pt-BR
PWA e web moderna	web.dev Learn: web.dev/learn e web.dev/learn/pwa
React	react.dev/learn
Next.js	nextjs.org/learn e nextjs.org/docs
TypeScript	typescriptlang.org/docs
Tailwind CSS	tailwindcss.com/docs
Supabase (banco, login, funções)	supabase.com/docs (procure "Row Level Security" e "Edge Functions")
PostgreSQL	postgresql.org/docs (Tutorial)
SQL interativo	sqlbolt.com
Git	Livro Pro Git (grátis, há versão em português): git-scm.com/book/pt-br/v2
GitHub Actions	docs.github.com/pt/actions
Vercel	vercel.com/docs
Segurança	OWASP Top 10: owasp.org/www-project-top-ten
Acessibilidade	WCAG 2.2 (guia rápido): w3.org/WAI/WCAG22/quickref
LGPD	ANPD: gov.br/anpd
Google Play	Ajuda do Play Console: support.google.com/googleplay/android-developer
IA: como pedir bem	Guia de engenharia de prompts da Anthropic (procure "Prompt engineering overview" em docs.anthropic.com , que pode ter mudado para docs.claude.com)

H.2 Cursos e trilhas gratuitos (escolha 1 ou 2, não todos)

- **CS50 (Harvard)**: cs50.harvard.edu/x/. Introdução à computação, ótima para a base. Há versões com legendas em português. Parte pesada: pode ver só as primeiras aulas.
- **freeCodeCamp**: freecodecamp.org. Exercícios práticos de web, com versão em português.
- **roadmap.sh**: roadmap.sh. Mapas visuais do que estudar (front-end, back-end, DevOps, produto). Use para saber "o que existe".
- **Nielsen Norman Group** (artigos sobre usabilidade): nngroup.com/articles. Comece por "10 Usability Heuristics".
- **Laws of UX**: lawsofux.com. Princípios curtos de experiência do usuário.
- **Material Design**: m3.material.io. Referência de tamanhos, toque e padrões em Android.

H.3 Livros (sugestões; confira edições em português e preços)

Livro	Assunto	Por que ler
Não Me Faça Pensar (Steve Krug)	Usabilidade	Curto e prático. Ensina a desenhar telas que não confundem.
O Design do Dia a Dia (Don Norman)	Design de produtos	Por que as pessoas erram em coisas "mal desenhadas" e como evitar.
The Mom Test (Rob Fitzpatrick; confira o título em português)	Conversar com usuários	Como perguntar sem ouvir só "gostei". Evita construir o que ninguém quer.
A Startup Enxuta (Eric Ries)	Produto e aprendizado	Construir pequeno, medir, aprender, ajustar.
Inspired (Marty Cagan)	Gestão de produto	Como pessoas de produto decidem o que construir.
Continuous Discovery Habits (Teresa Torres)	Descoberta contínua	Rotina semanal de falar com usuários e testar ideias.
Traction (Gabriel Weinberg e Justin Mares)	Conseguir clientes	19 canais para conquistar os primeiros usuários.
Shape Up (Ryan Singer, grátis em basecamp.com/shapeup)	Planejar e entregar	Como definir o escopo de uma entrega e não deixar crescer sem fim.
Refactoring UI (Adam Wathan e Steve Schoger; refactoringui.com)	Visual para quem não é designer	Dicas objetivas de espaçamento, cores e hierarquia.

Atomic Design (Brad Frost, grátis em atomicdesign.bradfrost.com)	Sistemas de design	Pensar em peças pequenas e reutilizáveis (como o <code>ui.tsx</code>).
Pro Git (Chacon e Straub, grátis)	Git	Referência completa de versionamento.
O Programador Pragmático (Hunt e Thomas)	Boas práticas	Hábitos de quem constrói software que dura.
Código Limpo (Robert C. Martin)	Qualidade de código	Para entender o que torna um código fácil de manter. Leitura opcional.
Designing Data-Intensive Applications (Martin Kleppmann)	Dados em escala	Avançado. Só quando o app crescer muito.
Co-Intelligence (Ethan Mollick)	Trabalhar com IA	Como pensar na IA como colega: onde ela ajuda e onde engana.

H.4 Perguntas para testar o que você entendeu

1. Explique, sem usar a palavra "banco", por que o app guarda dados no celular antes de mandar ao servidor.
2. Qual a diferença entre uma chave pública e uma secreta? Cite uma de cada no Medde.
3. O que é RLS e o que aconteceria sem ele?
4. Por que o painel pede os números a uma função e não lê o banco direto?
5. O que é uma migração e por que "esquecer de rodar uma" quebrou o painel?
6. O que significa o CI estar vermelho? Quando é aceitável publicar mesmo assim? (Resposta esperada: quase nunca.)
7. Escreva um pedido para uma funcionalidade nova usando a fórmula em 4 partes.
8. Quais decisões só você pode tomar e quais a IA pode sugerir?
9. Como você saberia se um pedido novo vai quebrar telas existentes?
10. Por que desenhar a tela antes de construir economiza tempo?

H.5 Glossário (A a Z)

Termo	Em palavras simples
API	Balcão por onde um programa pede serviço a outro.
aab / apk	Arquivos do app Android. O .aab vai para a loja; o .apk instala direto.
Authentication	Provar quem você é (login).
Authorization	O que você pode fazer depois de logado.
Backup	Cópia de segurança dos dados.

Branch	Cópia de trabalho do código, separada da versão oficial.
Build	Transformar o código em algo que roda (o app pronto).
CI / CD	Testar e publicar automaticamente a cada mudança.
Commit	Um ponto salvo do histórico, com mensagem.
Componente	Uma peça reutilizável da tela (botão, cartão).
CORS	Regra do navegador sobre quais sites podem chamar um servidor.
Deploy	Colocar uma versão no ar.
DNS	"Lista telefônica" que liga o nome do site ao servidor.
Edge Function	Pequeno programa que roda no servidor, perto do usuário.
Endpoint	Endereço específico de uma API ou função.
Env / variável de ambiente	Configuração que muda conforme o lugar onde o app roda.
Front-end / Back-end	O que o usuário vê / o que roda nos bastidores.
Funil	Etapas por onde o usuário passa (cadastro → uso → pagamento) e quantos chegam em cada uma.
Git / GitHub	Programa de histórico de versões / site que guarda esse histórico.
JSON	Formato de texto para trocar dados entre programas.
JWT	"Crachá" digital que prova que você fez login.
Lint	Corretor automático de estilo e erros simples do código.
Local-first	Guardar primeiro no aparelho e sincronizar depois.
Main	O branch oficial, que vai para os usuários.
MRR / Churn	Receita mensal recorrente / taxa de cancelamento.
Migração	Arquivo que muda o banco de forma controlada e repetível.
Mock	Versão falsa de um serviço, usada nos testes.
PWA	Site que se comporta como app.
Pull request	Pedido para juntar um branch ao oficial, com revisão.
RLS	Regra do banco: cada usuário só vê as linhas dele.
Rollback	Voltar para a versão anterior.
Sandbox	Ambiente de teste de um serviço (sem dinheiro real).

Secret	Valor confidencial guardado em local protegido.
Service worker	Ajudante do PWA nos bastidores (offline, notificações).
SQL	Linguagem para consultar e alterar bancos.
TWA	App Android que apenas abre o seu site em tela cheia.
Token	Código que dá acesso a algo (como uma senha temporária).
Webhook	Aviso automático de um serviço para o seu servidor.

Fechamento: o que levar daqui

Cinco hábitos que mais evitam retrabalho

1. **Desenhe antes de construir** e valide com 3 pessoas reais.
2. **Escreva as decisões** (e a razão) num lugar que sobreviva às conversas.
3. **Peça em fatias pequenas**, com "pronto quando" e prova.
4. **Nunca ignore o vermelho** (CI, rotinas, erros) e confira no aparelho real.
5. **Proteja segredos e dados**: chave nunca no chat; revogue se vazar.

Seu papel nos próximos apps

Você não precisa escrever o código. Precisa saber: **qual problema** resolver, **para quem**, **qual o menor passo**, **como saber que funcionou** e **o que não pode dar errado**. O resto, a IA constrói. E, quanto mais você entende da lógica, melhor você percebe quando ela errou.

Sugestão de próximo passo: escolha a próxima ideia de app e, antes de pedir qualquer coisa, preencha a fórmula da Parte D.1 e o ciclo da Parte E.1 até o passo 4 (desenho). Mande para a IA e peça: "critique este plano e me faça as perguntas que faltam".